INTERPRETABILITY · SAE FEATURE ANALYSIS · GEMMA-2-2B LAYER 13

Are residual-SAE features less "trivial"?

A two-day investigation into whether training a sparse autoencoder on the *residual* (activation minus a linear prediction from the token embedding) produces fewer single-token features than a normal SAE — and a methodological trap that made the answer look different depending on where I looked.

Notebook write-up   8 tests   Verdict: null (with caveats worth reading)

Bottom line up front

1. **Single-token ("trivial") features are rare — about 1%, not the ~16% the first metric suggested.** The 16% was a measurement artifact of only looking at each feature's *strongest* activations.
2. **Residualization does not reduce them.** Full and residual SAEs are equal on every metric that isn't peak-biased.
3. **Every apparent effect I found along the way** ("residual is more trivial," a ~22% feature "churn") turned out to be a *frequency* or *peak-sampling* artifact that vanished under proper controls.
4. The real deliverable is the **method**: how peak-bias and frequency confounds can manufacture effects that aren't there.

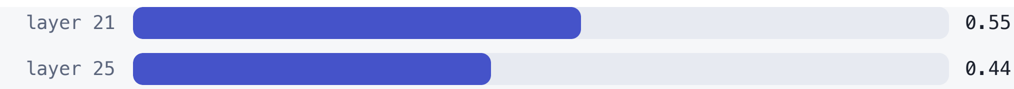
The setup & the hypothesis

I trained two sparse autoencoders (SAEs) on the layer-13 residual stream of Gemma-2-2b. Both are BatchTopK SAEs, 16,384 features, k=64, trained on 50M tokens with identical settings. The *only* difference is the input:

- **Full SAE** — trained on the raw activation `h`.
- **Residual SAE** — trained on `r = h − P(embedding)`, where `P` is a linear map I'd trained to predict the layer-13 activation from the token's embedding.

The idea: token identity is the "easy," linearly-predictable part of the activation. If I subtract it out, maybe the SAE stops wasting features on **trivial single-token detectors** (features that just fire on one word) and spends them on higher-level concepts instead.

The linear map explains a good chunk of the activation — here's how much variance it captures per layer (I used layer 13):



Test R^2 of the linear map (embedding → activation), by layer. Layer 13 (highlighted) sits in a sweet spot: enough token signal to remove, enough context left to matter.

Both SAEs reconstruct the true activation about equally well (the residual SAE's error on `r`, scaled back, matches the full SAE's error on `h` at ~0.14 FVU), so any difference between them isn't just one fitting better than the other.

How I measure "trivial"

A feature is **trivial** if it just fires on a single token — it's interpretable, but it's only re-detecting a word the embedding already told the model about. To measure it, for each feature I take its top activating examples and compute two things over the words they fire on:

- **modal fraction** — what fraction are the *same* word (high = single-token).
- **distinct words** — how many different words (low = single-token).

Words are normalized first, so `Happy` / `happy` / ` happy` count as one. This whole write-up is really the story of getting this one measurement right.

TEST 1 First pass — basic comparison

Did: Computed the trivial-feature fraction for both SAEs on 100k tokens.

Found: Basically identical — 2.7% vs 2.7%. Looked like a flat null.

> PROBLEM, FLAGGED BY MENTOR
>
> 100k tokens is far too small. A typical SAE feature fires on roughly 1 in 10,000–1,000,000 tokens, so at 100k I was only measuring the *most frequent* features and missing most of the dictionary. (Neuronpedia's dashboards use ~4M tokens.)

TEST 2 Better metric, much more data

Did: Cleaned up the metric (top-20 examples, string normalization, added the distinct-words measure) and scaled to **4M tokens** using a streaming method, so ~99.8% of features got measured.

Found: Still no clear difference in the distributions. If anything, the residual looked *slightly more* trivial in aggregate — 18% vs 16% pure single-word features — the opposite of the hypothesis.

TEST 3 Controlling for firing frequency

Did: Grouped features into firing-frequency bins and compared the two SAEs *within each bin* — because rare features are naturally more trivial, and the two SAEs might have different mixes of rare vs common features.

Found: The "residual is more trivial" from Test 2 was a **mirage**. It came entirely from the residual SAE having *more rare features* (which are intrinsically more trivial), not from its features being more trivial at any given frequency.

FINDING

At *matched* frequency, there is no real difference — the residual is even slightly *less* trivial in most bins. The naive aggregate comparison was misleading; once controlled, it's a clean null. **Lesson: frequency is a confound that can flip the sign of the result.**

TEST 4 Actually looking at the features

Did: Built a text "dashboard" (like a homemade Neuronpedia) showing, per feature: how often it fires, an activation histogram, which tokens it predicts, and its top examples with context. Then inspected specific features in both SAEs.

"district" feature

…U.S. **District** Court · Los Angeles County **District** Attorney · **District** of Columbia · a

Fires on "district" across totally different meanings (legal, governmental, geographic). It doesn't care about meaning, just the word. **Genuinely trivial** — a pure token detector. The metric got this one right.

"health" feature

…single-payer **health** care · lost their **health** insurance · universal **health** care · **hea**

Fires on "health" but specifically in *healthcare-policy* contexts, and it **predicts the token "insurance."** So it's a genuinely **meaningful feature** — yet the metric flags it as trivial because its top examples are all one word.

FINDING

"Single-token" ≠ "useless." The metric flags token-bound features, but some of them are real, useful concepts. Also: both SAEs have the same district and health detectors — so residualization didn't delete these concepts.

TEST 5 Matching features across the two SAEs

Did: The systematic version of Test 4. For 100 single-token features in one SAE, find the word each fires on, then find the feature that fires most on that word in the *other* SAE, and check if it's still single-token or has broadened. Did both directions, plus a **control matching each SAE against itself** to measure how much apparent "change" is just noise.

DIRECTION	BECAME MULTI-WORD	SELF-CONTROL (NOISE FLOOR)	REAL EFFECT
full → residual	30%	8%	~22%

DIRECTION	BECAME MULTI-WORD	SELF-CONTROL (NOISE FLOOR)	REAL EFFECT
residual → full	18%	6%	~12%

Subtracting the self-control noise floor: ~22% of the full SAE's single-token features broaden in the residual, and ~12% of the residual's are "new."

This looked like a real result: **feature churn** — the count of single-token features stays flat, but *which* concepts have them turns over by 12–22%. I was ready to write this up… and then Test 6 undercut it.

> THE CATCH — THIS USED THE "PEAK" METRIC
> This whole analysis judged triviality from each feature's *strongest* ~20 activations. Test 6 shows that's exactly the wrong place to look.

TEST 6 Looking across activation strength — the trap

Did: Instead of only the top activations, binned each feature's activations by *strength* (strongest fifth, next fifth, …) and checked triviality in each bin. A feature might be single-token at its peak but fire on many words when it's activating weakly.

Found: exactly that. Two examples, both labeled "single-token" by the top-20 metric:

feature that *is* robustly single-token

`bin 0 (strongest): order ×20 · bin 1: order ×20 · bin 2: order · bin 3: order · bin 4`

"order" all the way down. A **real** token detector.

feature that only *looks* single-token at the peak

`bin 0: news (but already 84 distinct words) · lower bins: rumors, False, Lied, Myths,`

Its top-20 are all "news," so the metric said single-token. But it's really a **misinformation / news-credibility concept feature** — broad and meaningful. The peak hid it.

Another one fired on "subscribing / signing / up / for" across bins — a **"sign up / subscribe" concept feature**, again mislabeled single-token from its peak.

> THE TRAP
> The top-20 metric is **peak-biased**: it labels a feature by its strongest activations, where almost everything looks token-specific. Several "trivial" features are actually the *higher-level* features I was hoping to find.

TEST 7 Fixing the metric — peak vs range vs weighted

Did: Measured triviality three ways over 4M tokens: on the **peak** (strongest firings), on a **uniform** sample of all firings, and on an **activation-weighted** sample (firings counted in proportion to how strongly the feature fired — the principled middle).

The headline: single-token fraction depends almost entirely on where you look

PEAK	WEIGHTED	UNIFORM
~16%	**~1%**	**0.3%**
strongest firings only	activation-weighted	all firings equally

Fraction of features that are pure single-word, same features, measured three ways. An ~80× swing.

METRIC	SINGLE-WORD (FULL)	SINGLE-WORD (RESID)	EFFECTIVE #WORDS (FULL)
Peak (strongest)	15.7%	17.9%	8.1
Weighted (principled)	**0.9%**	**1.5%**	**13.2**
Uniform	0.3%	0.2%	14.5

"Effective #words" = exp(entropy) of the word distribution — how many words the feature effectively responds to (1 = pure single-word). Out of 20.

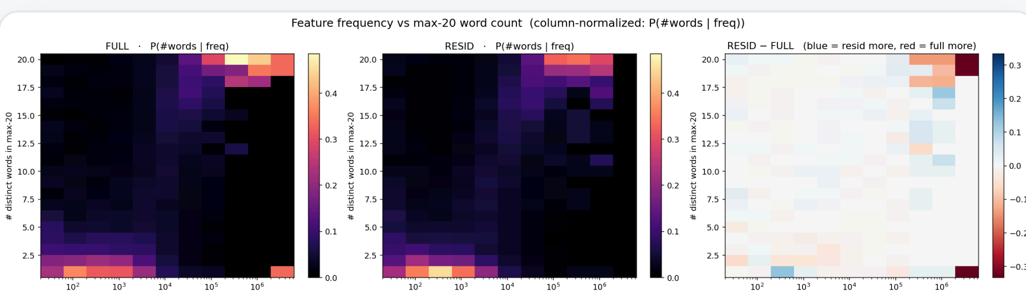
FINDING

Even the activation-*weighted* metric lands near the broad end (≈13 effective words, ~1% single-word), **not** halfway. So single-token-ness lives *only* at the razor-thin activation peak; the moment you weight in any of a feature's typical firing, it looks broad. **The ~16% was a peak artifact.**

And on the corrected metric, **full vs residual is null** — both ~1% single-word, ~13 effective words. The ~22% churn from Test 5 was a peak-behavior effect that disappears once you stop looking only at peaks. Within frequency bins it's mixed and tiny.

TEST 8 Frequency × complexity, as one heatmap

Did: Took the metric interpretability dashboards actually use — the number of distinct words in a feature's *max*-20 examples — and plotted it against firing frequency as a 2D heatmap, one panel per SAE. Each *column* is normalized, so it shows `P(#words | frequency)`: among features that fire this often, how many words do they spread over? A third panel shows residual – full. This folds the frequency control (Test 3) and the residual comparison into a single picture, with no arbitrary buckets.



Left / middle: P(#words | freq) for the full and residual SAEs (brighter = more mass). Right: residual − full (blue = residual has more mass there, red = full does). Firing frequency on a log x-axis, half-order-of-magnitude bins.

Found: one dominant structure — a diagonal **ridge from bottom-left to top-right**. Rare features (left) are pinned at 1–2 words; common features (right) spread across ~20. So **a feature's word-count is largely a function of how often it fires**, not of how abstract it is: a feature isn't "complex" because it encodes a rich concept — much of its word-count is simply "it fired enough times to have seen many words."

FINDING

This is the frequency confound as a *picture*: complexity tracks frequency, so any triviality comparison that doesn't hold frequency fixed is mostly re-measuring the frequency mix. And the **difference panel is near-white everywhere** — conditioned on frequency, full and residual have the same word-count distribution at every frequency. The residual null holds. (The faint blue in the rare corner = residual is a hair *more* single-token among rare features; the lone red cell at the far top-right is the ~2% common tail, too sparse to trust.)

Why the residual ever *looked* more trivial

If the two SAEs are identical at matched frequency, where did the aggregate "residual is more trivial" from Test 2 come from? From the **frequency mix itself.** Here is how many alive features each SAE places in each half-order-of-magnitude frequency bin:

FIRING FREQUENCY	FULL	RESID
20 – 63	176	351
63 – 200	542	955
200 – 632	1,123	1,793
632 – 2,000	2,463	2,206
2,000 – 6,325	3,408	2,796
6,325 – 20,000	5,124	4,897
20,000 – 63,246	3,132	2,996
63,246 – 200,000	322	276
> 200,000	59	79
total alive	16,349	16,349

Alive features per half-order-of-magnitude frequency bin. ~80% of each dictionary lives in the 632–63k range (peak bin: 6.3k–20k); the common tail (>63k) is only ~2%.

The tell is the **rare tail**. Below 632 firings the residual holds **3,099 features (19%) vs the full's 1,841 (11%)** — nearly double in the two rarest bins. Rare features are intrinsically single-token (they've simply seen fewer contexts), so a dictionary carrying more of them looks more trivial *on average* — even though, feature-for-feature at matched frequency, nothing has changed. That is the whole "residual is more trivial" effect: a **frequency-composition artifact**, not a complexity difference. (Why the residual's dictionary skews rarer is a separate question these tests don't pin down — plausibly, removing the token-embedding prediction frees the SAE from re-encoding common tokens — but the shift in the *mix* is real while the change in per-feature triviality is not.)

What I actually learned

1. **Trivial single-token features are rare (~1%), not ~16%.** The high number was a peak-sampling bias — features are token-specific at their strongest activation but broad in typical firing.
2. **Residualization doesn't reduce triviality.** Null on every metric that isn't peak-biased, and null within frequency bins.
3. **Every "effect" I found was an artifact.** The aggregate "residual is more trivial" was a frequency-composition artifact (Test 3); the 22% churn was a peak-sampling artifact (Tests 6–7).
4. **"Single-token" is not "useless."** Some token-bound features (health→insurance, the misinformation feature) are genuinely meaningful, so the framing of "trivial features are bad" needs rethinking anyway.

> THE METHODOLOGICAL TAKEAWAY
>
> Two confounds — **firing frequency** and **peak-sampling** — each independently manufactured an apparent effect that vanished under the right control (matched-frequency bins; activation-weighted measurement). The negative result is only trustworthy *because* those controls are in place.

Open questions & where I'd go next

- **Is "fewer single-token features" even the right goal?** The health/insurance and misinformation features suggest token-bound ≠ low-value.
- **Try an earlier layer** (e.g. layer 1, $R^2 \approx 0.85$), where residualization removes far more of the token signal — if any effect exists, it should be strongest there.
- **A non-linear map** instead of a linear one, so more token identity is removed from the residual (token identity clearly survives the linear subtraction — the residual SAE still learns a "district" detector).

Gemma-2-2b · L13 residual   2 SAEs · 16k feats · k=64 · 50M tok   eval on 4M tokens